

Hệ thống nhúng payload mang bằng chứng ZK vào video H.264/CAVLC

Tài liệu kỹ thuật dựa trên mã nguồn và artifact benchmark của dự án

05/10/2026

Tóm tắt (trạng thái ngày 05/10/2026). Hệ thống nhúng một payload chứa proof Groth16 vào bit dấu trailing-one của residual CAVLC trong các IDR H.264 Baseline, và phía nhận trích xuất mù chỉ với khóa giấu tin. Báo cáo mô tả phiên bản hiện tại: (1) chỉ còn *một lỗi media native C++*; pipeline Python song song cũ và benchmark SEC1–SEC10 đã gỡ ngày 03/10/2026; (2) kênh nhúng dùng *giao thức v3* (04/10/2026): khung $[0x03][len\ 2\ B][payload]$ *không có MAC*, hai khóa con HKDF-SHA256 (lịch nhúng và làm trắng bit), lịch theo đoạn IDR với mặc định 64 bit/IDR; (3) *proof camera*: mạch `circuits/camera_video.circom` (8,737 constraints, 3 public input root, `bindingHi`, `bindingLo`) chứng minh một camera nằm trong sổ đăng ký Poseidon Merkle đã xác nhận *đúng video này* (hash toàn video trừ các bit carrier) và *đúng message này*; bên kiểm chứng chỉ cần root của sổ đăng ký, verification key và khóa giấu tin, không cần secret camera và không biết camera nào; (4) bằng chứng hiện tại: 142/142 test qua 12 phase và CTest native 1/1; suite benchmark ghi ngày 04/10/2026 (i7-12700H, Windows 11) cho media 27/27, đầu-cuối 5/5 và ma trận 11 kiểu tần công đều đúng kỳ vọng. Số đo camera vật lý (7.608 FPS) vẫn là số liệu lịch sử của giao thức v1 và không đạt công 30 FPS.

1 Kiến trúc hệ thống

Hệ thống có ba lớp với trách nhiệm tách biệt. Lớp *proof* (Python + circom/snarkjs) tạo proof Groth16 cho phát biểu “một camera trong sổ đăng ký xác nhận video và message này”. Lớp *kênh giấu tin* đóng gói payload vào một khung v3 (không MAC), làm trắng bit và quyết định vị trí nhúng theo khóa giấu tin K . Lớp *media* (native C++, `native/src/cavlc_stream.cpp`) phân tích bitstream H.264 Baseline/CAVLC, liệt kê vị trí mang bit, lật dấu hệ số mà không đổi độ dài mã và tính hash video. Phía nhận chỉ cần video stego, K , root của sổ đăng ký và verification key: trích xuất mù, tính lại hash video từ chính file nhận được, rồi bắt buộc xác minh Groth16. Kênh không tự xác thực payload; chỉ proof làm việc đó.

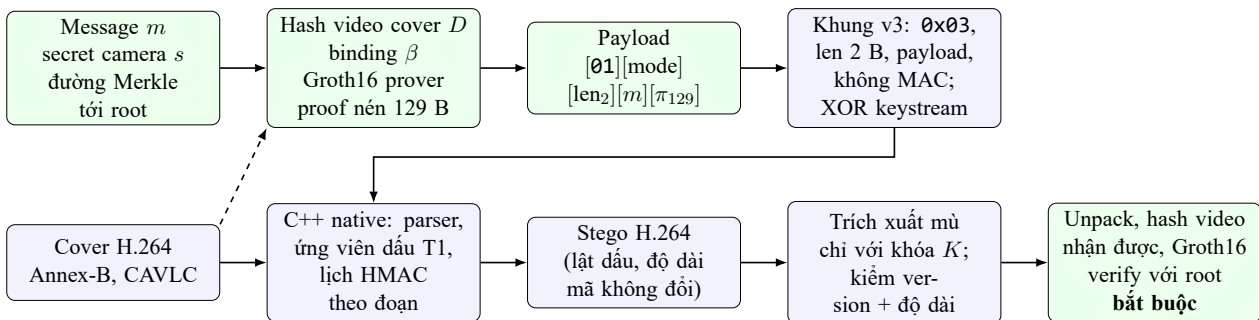


Figure 1: Các khối ứng dụng và video của hệ thống hiện tại. Mũi tên nét đứt: phía gửi tính hash của cover trước khi tạo proof. Dữ liệu được nhúng trong residual H.264 của IDR; hệ thống không dùng SEI hay container làm nơi chứa payload.

Hình 1 phân biệt proof với carrier: Groth16 tạo bằng chứng một lần cho cặp (video, message), còn C++ chỉ xử lý bitstream và không chạy prover trong vòng lặp camera. Hash video bỏ qua đúng các bit carrier nên giá trị tính trên cover và trên stego bằng nhau; mọi thay đổi khác của video làm proof sai (mục 7). Cùng một giao thức theo đoạn được dùng cho file, job HTTP và luồng WebSocket. Tìm thấy khung hợp lệ chỉ có nghĩa

là có dữ liệu ở đúng vị trí theo khóa; payload trích ra mang nhãn **"verified": false** cho tới khi proof được xác minh.

Table 1: Thành phần và hợp đồng dữ liệu chính (mã nguồn tại thời điểm 05/10/2026).

Khối	Đầu vào	Đầu ra / trách nhiệm
Sổ đăng ký camera (src/camera_registry.py)	Khóa công khai $pk_i = \text{Poseidon}(s_i)$	Cây Merkle Poseidon độ sâu 16 (tối đa 65,536 camera); bên kiểm chứng chỉ tin root.
Hash video (video-digest, src/video_binding.py)	Video, khóa K , số bit khung, cap	SHA-256 của toàn bộ NAL với đúng các bit carrier đặt về 0; cover và stego cho cùng giá trị.
Circom/snarkjs (src/zk_proof.py, src/camera_proof.py)	s , đường Merkle, root, binding	Proof Groth16 BN254 nén 129 byte; verify chỉ chấp nhận khi snarkjs thoát 0 và in OK!.
Payload packer	Mode, message, proof bytes	[0x01][mode][len 2 B][message][proof 129 B].
Khung kênh v3	Payload, khóa K 32 byte	HKDF tách hai khóa con; khung [0x03][len 2 B][payload], không MAC; bit khung XOR keystream.
Annex-B/NAL reader	Byte stream H.264	NAL unit, header, EBSP/RBSP; reader tăng dần nhận start code vắt qua chunk.
CAVLC parser + bộ thu ứng viên	IDR I-slice Base-line/CAVLC	Mỗi khối residual một ứng viên: dấu trailing-one đầu tiên, kèm offset bit RBSP.
Lịch + patcher theo đoạn	Ứng viên, bit đã làm trắng, khóa lịch	Chọn tối đa <code>max_bits_per_IDR</code> vị trí mỗi đoạn SPS+PPS+IDR, sửa bit dấu, dựng lại EBSP/Annex-B.
Trích xuất mù + verify	Stego, K , root, verification key	Payload chưa xác thực; job verify unpack, tính hash video của file nhận rồi Groth16 verify bắt buộc.

Ngoài các khối trên, `src/native_blind_contract.py` là bản tham chiếu Python của lịch/khung và `src/video_binding.py` là bản tham chiếu của hash video (cả hai có test đối chiếu với C++), không phải một đường media thứ hai.

2 Benchmark và kết quả chính

Phần này được đưa lên trước phần giải thích mã hóa để người đọc biết ngay điều gì đã chạy được và bằng chứng đang ở trạng thái nào.

2.1 Phạm vi bằng chứng (cập nhật 05/10/2026)

Table 2: Nguồn bằng chứng và mức độ áp dụng cho phiên bản hiện tại.

Nguồn	Thời điểm	Áp dụng cho bản hiện tại
Bộ test run_all.py (12 phase) và CTest native	04/10/2026	Có, về chức năng: 142/142 test và CTest 1/1 qua trên host Windows; không gồm phase camera vật lý H1; không phải nghiệm thu edge.
Media 20261004T142448Z_1db00a	run 04/10/2026	Có: giao thức v3, proof camera thật, mọi proof ràng buộc video.
e2e_new.json (5 lần chạy + ma trận tấn công), zkp_new.json	04/10/2026	Có: cùng giao thức và mạch camera_video.
Camera vật lý realtime_camera_runs_20260924.json (run 24)	24/09/2026	Không: số liệu lịch sử, giao thức v1, mạch cũ, trước bản sửa bộ đọc stdin native; chưa đo lại.
Benchmark của pipeline Python cũ	SEC1-SEC10 trước 03/10/2026	Không: đã gỡ; chỉ còn trong lịch sử git, không được trích dẫn.

Host đo: Intel Core i7-12700H (14 nhân vật lý, 20 luồng), 25.4 GB RAM, Windows 11, Python 3.12.10, FFmpeg 8.0.1, Node.js 22.20.0, snarkjs 0.7.6; JSON ghi commit và hash SHA-256 của các tệp native đã dùng.

2.2 Thiết kế phép đo media (suite_new, 04/10/2026)

Tập thử gồm 9 clip Y4M CIF (Akiyo, City, Coastguard, Container, Deadline, Football, Foreman, Foreman 300-frame, Hall Monitor), mỗi clip ở 352×288 , 640×480 và 1280×960 : 27 cấu hình, chỉ 30 frame đầu mỗi cấu hình. Encoder là FFmpeg/libx264 Constrained Baseline, CAVLC, QP 22, GOP=1/all-intra, không B-frame, một thread và một slice/IDR. Hai cỡ lớn được phóng từ nguồn CIF nên là stress test phân giải, không phải nội dung camera native HD. Chỉ foreman_cif.y4m được lưu trong repository, nên clone mới tái tạo 3 thay vì 27 cấu hình.

Mỗi cấu hình dùng một khóa giấu tin 32 byte mới; một camera trong sổ đăng ký 8 camera tạo proof Groth16 thật ràng buộc hash của cover và message 68 byte. Payload $4 + 68 + 129 = 201$ byte, khung $3 + 201 = 204$ byte, tức $8 \cdot 204 = 1,632$ bit, trải trên $\lceil 1632/64 \rceil = 26$ IDR ở mặc định 64 bit/IDR. Một cấu hình chỉ qua khi đủ mọi điều kiện: quét capacity native, embed native, FFmpeg strict decode (-xerror), trích xuất mù đúng payload, proof ràng buộc video verify được với root (hash tính lại từ file stego), khóa sai bị từ chối, và không có dấu nào đổi ngoài lịch (kiểm từng vị trí bằng zkstego_inspect --segments).

Table 3: Chi phí native và chất lượng theo độ phân giải (run 20261004T142448Z_1db00a). Median trong từng nhóm 9 clip.

Chỉ số	352×288	640×480	1280×960
Quét capacity mỗi IDR (ms)	52	220	770
Embed mỗi IDR mang khung (ms)	61	245	891
Hash video, 26 IDR carrier (s)	1.6	6.4	22.6
Output FPS (30 frame / thời gian embed)	18.94	4.71	1.30
Peak RSS tiến trình native embed (MiB)	11.44	20.55	54.55
PSNR-Y source-to-stego trung bình clip (dB)	43.73	45.08	46.59
Số cấu hình qua gate	9/9	9/9	9/9

Chi phí native tăng theo số IDR phải parse và theo số macroblock mỗi IDR. Thời gian embed toàn clip xấp xỉ thời gian quét capacity toàn clip; hash video parse cùng 26 IDR mang khung nên tốn thời gian tương đương embed. Groth16 prove/verify ($\approx 1.5\text{--}1.8\text{ s}$ và $\approx 0.9\text{--}1.0\text{ s}$) gần như không đổi theo độ phân giải, nên *parse bitstream*, không phải *Groth16*, là yếu tố giới hạn throughput. Output FPS chỉ là 30 frame chia thời gian embed của một lần chạy offline; nó không gồm capture camera hay encode và không phải phép đo realtime. RSS lấy mẫu mỗi 5 ms nên có thể hụt spike ngắn.

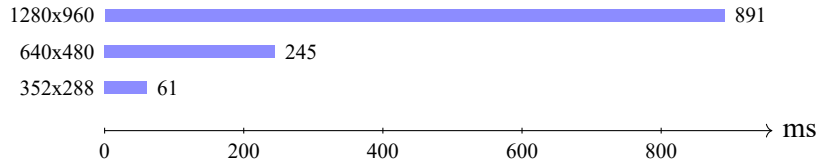


Figure 2: Median thời gian embed native trên mỗi IDR mang khung theo độ phân giải (27 cấu hình, giao thức v3, 64 bit/IDR).

Đây là phép đo embed offline trên đoạn H.264 đã encode, không phải đo end-to-end từ cảm biến. Ở 352×288 một IDR mất khoảng 61 ms, nghĩa là xử lý tuần tự từng IDR chưa đạt 30 IDR/s; ở 1280×960 khoảng 0.9 s mỗi IDR. Kết quả không hỗ trợ kết luận đạt realtime 30 FPS ở bất kỳ độ phân giải nào trong cấu hình đo.

Table 4: Tóm tắt chất lượng ảnh và kiểm tra chức năng (run 20261004T142448Z_1db00a, giao thức v3).

Đại lượng	Kết quả	Cách đọc
Gate chức năng	27/27	Mọi proof ràng buộc video và verify được.
Dấu bị đổi ngoài lịch	0	Kiểm từng vị trí trên cả 27 cấu hình.
Trung bình PSNR-Y source-to-stego	46.70 dB	Gồm cả nén H.264 lossy và nhúng.
Trung bình SSIM source-to-stego	0.99045	Trung bình trên 27 cấu hình.
PSNR-Y cover-to-stego thấp nhất mỗi cấu hình	44.03–58.68 dB	Thấp nhất: Container, 640×480 .
Cấu hình có frame dưới 40 dB	0/27	40 dB là mốc tham khảo, không phải gate chức năng.
Frame bị đổi mỗi cấu hình	26/30	Đúng 26 IDR mang khung; 4 frame cuối giống hệt cover.
Dấu bị lật / vị trí trong lịch	776–876 / 1,632	Khoảng một nửa, do bit làm trắng giả ngẫu nhiên.
Thống kê dấu (tỉ lệ trailing-one âm)	$ z \leq 0.22$	Kiểm định hai tỉ lệ cover so với stego.
Kích thước stego	bằng cover	Không đổi số byte trong mọi cấu hình.
Quét capacity toàn clip	median 5.94 s; max 31.14 s	Tách biệt với thời gian embed.

PSNR-Y đo sai khác cường độ luma theo dB; SSIM gần 1 khi cấu trúc hai ảnh tương tự. Source-to-stego gộp hai nguồn sai khác (nén và nhúng), còn cover-to-stego đối chiếu bản H.264 sạch với bản stego để cô lập ảnh hưởng nhúng. Với 64 bit/IDR, dấu bị lật trải trên 26 frame thay vì dồn vào một frame như các run cũ dùng 4096 bit/IDR; nhờ vậy frame tệ nhất vẫn trên 44 dB. Chỉ số $|z|$ nhỏ là kiểm tra thống kê bậc một, không phải thực nghiệm steganalysis.

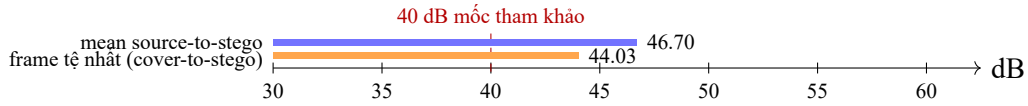


Figure 3: PSNR-Y trung bình source-to-stego so với frame bị nhúng tệ nhất trên toàn bộ 27 cấu hình (giao thức v3).

Hai đại lượng không cùng tham chiếu: 46.70 dB là trung bình gồm cả nén, còn 44.03 dB là frame tệ nhất do riêng bước nhúng. Sai khác do nhúng ở frame xấu nhất vì vậy cùng bậc với sai khác do nén ở QP 22; không cấu hình nào có frame dưới mốc 40 dB, nhưng trung bình cả clip vẫn che frame xấu nên cần xem giá trị theo từng frame trong `quality_per_frame_new.csv`.

2.3 Đầu-cuối, ma trận tấn công và proof system

Table 5: Thời gian từng bước đầu-cuối, median của 5 lần chạy trên clip CIF 300 frame (`e2e_new.json`, 04/10/2026). Clip có 300 đoạn IDR, 188,556 dấu ứng viên, dung lượng 19,200 bit ở 64 bit/IDR.

Phía	Bước	Median
Gửi	Hash video của cover	0.51 s
Gửi	Groth16 prove	1.77 s
Gửi	Embed native	0.50 s
—	FFmpeg strict decode (kiểm tra)	0.15 s
Nhận	Trích xuất mù	0.53 s
Nhận	Verify (hash video của stego + Groth16)	1.45 s
Tổng	Phía gửi / phía nhận	2.78 s / 1.99 s

Mỗi lần chạy dùng khóa và message mới (ví dụ message 71 byte $\Rightarrow$ payload $4 + 71 + 129 = 204$ byte $\Rightarrow 8 \cdot (3 + 204) = 1,656$ bit khung, 26 IDR). Dung lượng của clip ở 64 bit/IDR là $300 \cdot 64 = 19,200$ bit, tức payload tối đa $19200/8 - 3 = 2,397$ byte và message tối đa $2397 - 133 = 2,264$ byte. Cả 5/5 lần chạy qua strict decode, trích xuất đúng và proof rằng buộc video; 0 dấu đổi ngoài lịch.

Table 6: Ma trận tấn công qua quyết định của job verify (e2e_new.json): 11/11 trường hợp cho đúng kết quả kỳ vọng.

Trường hợp	Thao tác	Kết quả (lý do)
baseline	Stego gốc, đúng khóa, đúng số đăng ký	chấp nhận
wrong_key	Khóa giấu tin sai	từ chối (payload_not_found)
flip_scheduled_sign	Lật 1 dấu nằm trong lịch (giữa khung)	từ chối (malformed_proof_payload)
flip_unscheduled_carrier_sign	Lật 1 dấu ứng viên ngoài lịch trong IDR mang khung	từ chối (proof_invalid)
flip_later_frame_sign	Lật 1 dấu ở IDR cuối, ngoài vùng nhúng	từ chối (proof_invalid)
drop_first_idr	Xóa IDR mang bit đầu của khung	từ chối (payload_not_found)
truncate_half	Cắt còn 50% đầu stream (payload còn nguyên)	từ chối (proof_invalid)
transcode	Giải mã rồi mã hóa lại (libx264, QP 22)	từ chối (payload_not_found)
message_swap	Người có khóa giấu tin thay message, giữ proof	từ chối (proof_invalid)
unregistered_camera	Camera ngoài sổ đăng ký tự tạo proof với cây riêng	từ chối (proof_invalid)
replay_other_video	Chép payload sang video khác (Akiyo) cùng khóa	từ chối (proof_invalid)

Ma trận cho thấy phát lại sang video khác, cắt ngắn, sửa frame sau vùng nhúng, lật dấu không mang khung, mã hóa lại, thay message, camera ngoài sổ đăng ký và sai khóa đều bị từ chối. Các trường hợp bị chặn ở bước trích xuất (payload_not_found) cho thấy kênh mỏng manh có chủ đích: bất kỳ lần mã hóa lại nào cũng phá payload.

Table 7: Groth16 và PLONK trên cùng RICS camera_video (8,737 constraints, 3 public input) và cùng witness; Groth16 5 lần, PLONK 3 lần (zpk_new.json, 04/10/2026).

Chỉ số	Groth16 BN254	PLONK KZG	Diễn giải
Witness	≈170 ms	≈172 ms	Có thời gian khởi động Node.
Prove	1.52–1.60 s	21.5–23.3 s	PLONK chậm hơn khoảng 14 lần.
Verify	0.91–1.01 s	0.90–0.94 s	Phần lớn là khởi động Node/snarkjs.
Peak RSS prover	≈0.73 GB	≈0.89–0.91 GB	Lấy mẫu mỗi 10 ms, có thể hệt peak ngắn.
Proof serialized	129 B nén / ≈805 B JSON	≈2,250 B JSON	Bộ nén 129 byte chỉ có cho Groth16.
Setup/artifact	Phase 1 Hermez 2 ¹⁶ + Phase 2 cục bộ (demo); proving key 4.2 MB	setup 3.2 s; proving key 30.1 MB	PLONK dùng cùng Powers of Tau công khai làm SRS phổ quát.
Public input bị sửa	bị từ chối	bị từ chối	Kiểm tra âm trong mọi lần thử.

Groth16 có proof nhỏ nhất và prove nhanh hơn rõ rệt; đó là lý do payload dùng Groth16. Mạch SHA-256 cũ (63,321 constraints, nay là lịch sử) cần 3.5–4.4 s để prove; mạch Poseidon/Merkle hiện tại nhỏ hơn khoảng 7 lần. So sánh không gồm Halo2, STARK/RISC Zero hay Bulletproofs vì chưa có triển khai cùng mạch trong repository (cần toolchain Rust).

2.4 Camera vật lý (số liệu lịch sử, giao thức v1)

Table 8: Một lần chạy camera E2E 60 giây (run 24, 24/09/2026). Số liệu lịch sử, giao thức v1 và mạch cũ, không phải bằng chứng hiện tại; chưa đo lại với giao thức v3 và proof camera.

Đại lượng	Kết quả	Cách đọc
Frame ghi/giải mã	450 / 450	FFmpeg strict decode thành công.
Throughput	7.608 FPS active; 7.331 gồm khởi động	Thấp hơn công 30 FPS: không đạt.
Proof và payload	Proof 129 B; verify đúng	Proof tạo trước capture, verify sau trích xuất.
Negative checks	Khóa sai và message sửa bị từ chối	Xác nhận trong lần chạy đó.
PSNR-YUV420 toàn video / frame sửa thấp nhất	65.23 / 46.78 dB	Mean SSIM luma 0.999928.
Process-tree peak RSS	188.84 MB	Gồm API, FFmpeg và native; lấy mẫu mỗi 100 ms.

Run 24 cho thấy, ở giao thức v1, đường camera (DirectShow $352 \times 288 \rightarrow$ FFmpeg $\rightarrow$ WebSocket loopback TCP $\rightarrow$ native) đã truyền proof thật, trích xuất mù và verify sau capture. Một chẩn đoán riêng trên cùng host cho thấy trong điều kiện đó chính nguồn camera chỉ cấp khoảng 7.5 frame/s, nên con số 7.608 FPS chưa quy được cho transport hay xử lý native. Đây là một run đơn trên host phát triển; nó không chứng minh ổn định lâu dài, realtime 30 FPS hay kết quả trên thiết bị edge.

Ghi chú hiệu năng. Ngày 02/10/2026 bộ đọc stdin và bộ đọc bit native được sửa (reader không chờ lấp đầy khối lớn trên pipe live, quét start code tuyến tính). Một lần embed live 1080p all-intra trên host phát triển giảm từ 30.1 s xuống 2.5 s. Đây là phép đo chức năng, không phải benchmark có kiểm soát; con số 7.608 FPS được đo *trước* bản sửa và chưa được đo lại.

3 Từ ảnh đến H.264: nhìn thấy từng bước

3.1 RGB, Y, Cb, Cr và lấy mẫu màu

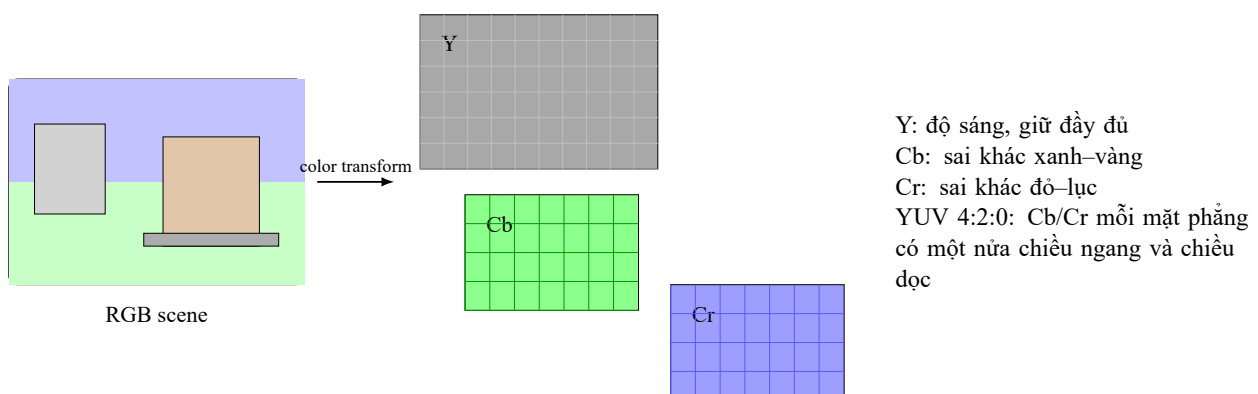


Figure 4: Minh họa tự dựng RGB được phân thành ba mặt phẳng. Độ phân giải mỗi mặt phẳng màu thực tế phụ thuộc pixel format; benchmark dùng YUV420.

RGB mô tả màu theo ba thành phần đỏ, lục, lam; encoder thường chuyển sang biểu diễn luma/chroma. Với YUV420, mỗi mẫu Cb/Cr đại diện vùng 2×2 pixel, vì mắt nhạy với độ sáng hơn sai khác màu. Frame 352×288 do đó giữ mặt phẳng Y 352×288 , còn mỗi chroma plane là 176×144 (một frame Y4M 4:2:0

gồm $101,376 + 25,344 + 25,344 = 152,064$ byte dữ liệu). Hình này là sơ đồ giải thích, không phải frame benchmark.

3.2 Macroblock, dự đoán và residual

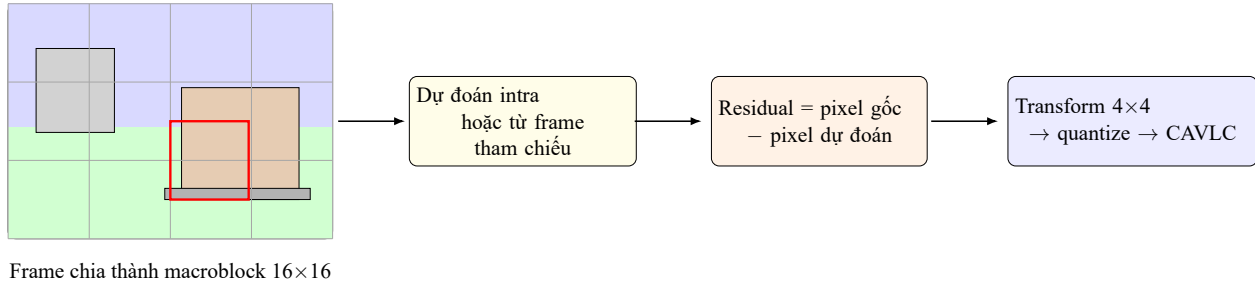


Figure 5: Macroblock là vùng ảnh được mã hóa cùng thông tin dự đoán và residual; ô đỏ là ví dụ vùng được phóng to.

Encoder không biến thẳng pixel thành bit. Nó hình thành dự đoán từ các mẫu *đã tái tạo* (trước deblocking) của khối lân cận, trừ dự đoán khỏi block nguồn để lấy residual, biến đổi residual thành hệ số tần số, lượng tử hóa và cuối cùng entropy-code. Với all-intra, frame không tham chiếu frame trước; đường nhúng native chỉ mang dữ liệu trong IDR I-slice/CAVLC. Vì dự đoán intra dùng mẫu đã tái tạo, một lần lật dấu có thể lan truyền sai khác sang các khối giải mã sau trong cùng frame.

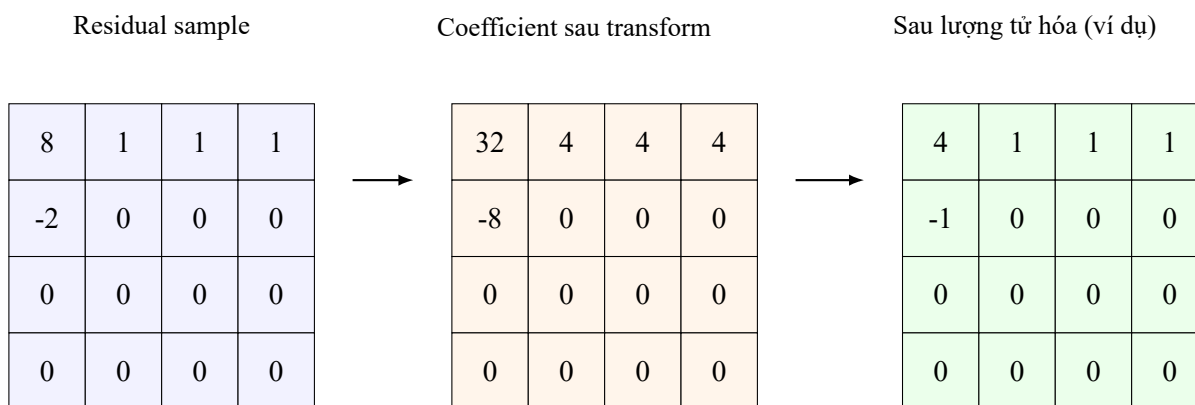
3.3 Lượng tử hóa và bảng QP

H.264 không bắt buộc một ma trận quantization duy nhất cho mọi block. QP xác định mức bước lượng tử hóa; scaling lists (nếu có) điều chỉnh theo tần số và loại block. Dưới đây là chu kỳ hệ số bước cơ sở chuẩn hóa theo QP, với $q = \lfloor QP/6 \rfloor$, $r = QP \bmod 6$, và $Q_{step} = V_r 2^q$; hệ số V_r theo quan hệ QP trong [ITU-T H.264 phiên bản 16](#). Đây là mô hình bước lượng tử để đọc QP, không thay thế phép biến đổi/rounding integer chính xác ở encoder.

Table 9: Hệ số bước lượng tử cơ sở theo phần dư QP modulo 6.

$r = QP \bmod 6$	V_r	QP mẫu	Q_{step} chuẩn hóa
0	0.6250	0	0.6250
1	0.6875	1	0.6875
2	0.8125	2	0.8125
3	0.8750	3	0.8750
4	1.0000	4	1.0000
5	1.1250	5	1.1250

Khi QP tăng 6, bước cơ sở tăng gấp đôi; hệ số nhỏ dễ bị lượng tử về 0. Tham số encoder là `-qp 22` ($q = 3$, $r = 4$, bước cơ sở $1 \times 2^3 = 8$). Tuy vậy libx264 ở chế độ QP cố định hạ QP của I-frame theo `ip_ratio=1.40`: trace slice header của demo cho QP slice = $22 - 3 = 19$. QP thực của từng slice/MB vì vậy phải đọc từ bitstream, không suy từ tham số dòng lệnh. Hệ số được sửa để nhúng nằm *sau* lượng tử hóa, vì thay đổi trước bước này sẽ bị encoder ghi đè.



Số liệu được dùng để minh họa phép chia và làm tròn, không phải trace từ file encode. Trong H.264 thật, encoder áp dụng integer transform, scaling factor, shift và rounding theo vị trí (libx264 còn dùng dead-zone, trellis, psy-RD); vì vậy không dùng phép chia 8 giản lược để tái tạo bit-exact.

Figure 6: Ví dụ số học minh họa: một residual block tạo ra hệ số biến đổi, sau đó lượng tử hóa làm hệ số nhỏ trở thành 0.

Lượng tử hóa làm thưa ma trận coefficient để entropy coder mã hóa ít symbol hơn, đổi lại mất thông tin không khôi phục được. Hệ thống sửa dấu coefficient đã lượng tử hóa và không thay QP; vì vậy nó không thể khôi phục chi tiết đã mất từ bước nén.

3.4 Cấu hình encode tái lập

Benchmark gọi Ffmpeg/libx264 trên Y4M (benchmark/media_benchmark_new.py). Phần chính của lệnh như sau:

```
ffmpeg -i input.y4m -frames:v 30 \
-vf scale=W:H:flags=lanczos,format=yuv420p \
-c:v libx264 -preset ultrafast -tune zerolatency \
-profile:v baseline -qp 22 -g 1 -bf 0 -threads 1 -pix_fmt yuv420p \
-x264-params "keyint=1:min-keyint=1:scenecut=0:repeat-headers=1:slices=1:threads=1" \
-f h264 output.h264
```

Các flag quyết định điều kiện benchmark, không phải quy tắc của mọi H.264. Constrained Baseline dùng CAVLC; -g 1 -bf 0 tạo all-intra không B-frame; một thread/một slice là bắt buộc vì parser native chỉ nhận một slice mỗi frame bắt đầu ở MB 0 (multi-slice tự động của Ffmpeg nằm ngoài hợp đồng hỗ trợ). GOP>1 vẫn xử lý được nhưng P-frame không mang ứng viên.

4 NAL, slice và loại frame I/P/B

4.1 Từ Annex-B bytes đến NAL header

Annex-B là cách đóng khung byte stream H.264 bằng start code 3 hoặc 4 byte; NAL parser là phần mềm tìm marker, tách NAL, đọc header và chuyển EBSP/RBSP. Đầu stream ví dụ: 00 00 00 01 67 ... (SPS), 00 00 00 01 68 ... (PPS), 00 00 01 06 ... (SEI của x264), 00 00 01 65 ... (IDR slice). Byte header có forbidden_zero_bit, nal_ref_idc, nal_unit_type; type 5 nhận diện IDR slice. Start code không phải là header NAL.

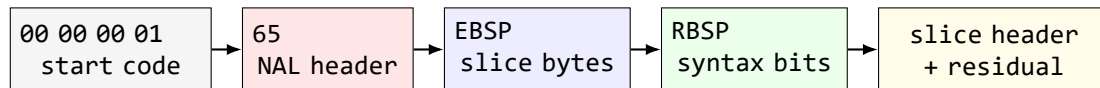


Figure 7: Giải phân NAL IDR minh họa. 0x65 có NAL type 5; payload sau header cần unescape trước khi đọc syntax.

Parser tìm start code, lấy NAL header, rồi bỏ byte emulation-prevention 03 để khôi phục Rbsp; mọi offset bit của ứng viên tính trên Rbsp. Lật một bit Rbsp có thể tạo hoặc xóa mẫu 00 00 0x, nên khi ghi lại NAL các byte EPB được tính lại: độ dài bit Rbsp không đổi nhưng độ dài byte EBSP có thể lệch vài byte. Trong streaming, `AnnexBNalStreamReader` giữ tối đa một NAL cộng một chunk đầu vào, nhận start code vắt qua ranh giới chunk và từ chối NAL vượt giới hạn cấu hình.

4.2 I/P/B không đồng nhất với NAL type

I, P, B là kiểu slice/dự đoán; IDR là điểm làm mới trạng thái tham chiếu được báo bởi loại NAL. Slice header có trường `slice_type`; giá trị modulo 5 cho biết P/B/I/SP/SI. Ví dụ `slice_type=2` là I, còn `slice_type=7` cũng là I (mọi slice trong ảnh cùng loại). Một I-slice không nhất thiết là IDR; P/B-slice thường nằm trong NAL type 1.

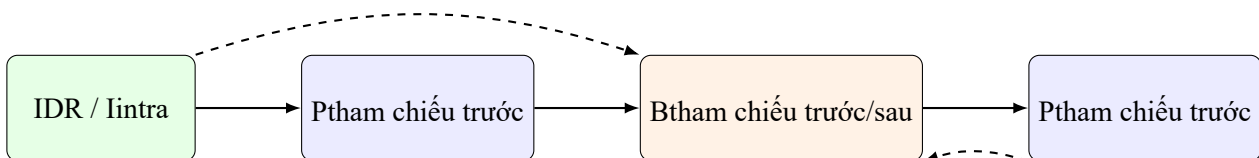


Figure 8: Ví dụ quan hệ tham chiếu theo thứ tự hiển thị, chỉ minh họa khái niệm. Benchmark dùng GOP=1 nên không có chuỗi P/B; hệ thống không hỗ trợ B-frame.

4.3 Hàm parser trong lõi native

Hệ thống hiện chỉ có một parser media: lõi C++ trong `native/src/cavlc_stream.cpp` (giao diện `native/include/zkstego/cavlc_stream.hpp`). Nó tách Annex-B, ghi nhớ đúng một SPS và một PPS, lọc và giải mã IDR I-slice, rồi thu offset dấu của trailing-one. Parser nhận Baseline, CAVLC, progressive, 4:2:0, một slice/frame bắt đầu ở MB 0, không I_PCM, không nhiều SPS/PPS khác id; cú pháp ngoài phạm vi bị từ chối thay vì đoán.

```

// native/src/cavlc_stream.cpp (ướcl ầphn ểkim tra ốli)
split_annex_b(annex_b)
inspect_baseline_idr_headers(annex_b)
decode_baseline_i_idr_slices(annex_b)
collect_cavlc_trailing_one_sign_candidates(slices)
patch_annex_b_nal_rbsp(annex_b, nal_index, patches)
auto units = split_annex_b(annex_b);
for (const auto& nal : units) {
    const auto rbsp = nal.rbsp();
    if (nal.nal_unit_type == 7) parse_baseline_sps(rbsp);
    else if (nal.nal_unit_type == 8) parse_baseline_pps(rbsp);
    else if (nal.is_idr()) decode_baseline_i_idr_slice(rbsp, sps, pps);
}
  
```

Đoạn mã là sơ đồ rút gọn theo trình tự thật: giữ SPS/PPS để diễn giải slice, chỉ đi vào decoder với IDR. Hàm thực tế còn kiểm tra PPS/SPS tồn tại và parse header trước khi đọc residual macroblock; parse thất bại nghĩa là bitstream không thuộc input được hỗ trợ. Công cụ soi duy nhất `zkstego_inspect` xuất NAL/slice/ứng viên dạng JSON (`--summary`, `--macroblock NAL MB`, `--segments MAX_BITS`); demo đối chiếu header với bộ `trace_headers` độc lập của FFmpeg.

5 Residual 4×4 , CAVLC và vị trí nhúng

Một residual block 4×4 sau transform/quantization là ma trận coefficient thưa. CAVLC quét theo zig-zag và mã hóa theo thứ tự: `coeff_token` (cặp `TotalCoeff`, `TrailingOnes`; bảng VLC chọn theo ngữ cảnh `nC`), `trailing_ones_sign_flag` (một bit dấu cho mỗi trailing-one, $0 \rightarrow +1$, $1 \rightarrow -1$), `level`, `total_zeros` và `run_before`. Kênh nhúng là bit `trailing_ones_sign_flag`.

Coefficient trước

Sau nhúng bit

4	1	0	0
0	-1	0	0
0	0	0	0
0	0	0	0

→

4	-1	0	0
0	-1	0	0
0	0	0	0
0	0	0	0

Bit nhúng đặt dấu $+1 \leftrightarrow -1$; magnitude giữ nguyên. Đây là ví dụ sign-bit, không phải toàn bộ codeword CAVLC.

Figure 9: Minh họa một sign bit trong ma trận coefficient thưa. Bộ giải mã CAVLC cần giữ chính xác cấu trúc token và offset bit.

Lật bit dấu trailing-one có ba tính chất được hệ thống dựa vào: (i) độ dài mã không đổi, mọi phần tử sau giữ nguyên vị trí; (ii) `TotalCoeff` và `TrailingOnes` không đổi nên `nC` của các khối sau và bảng VLC được chọn không đổi, bitstream vẫn hợp lệ; (iii) thay đổi ở mức lượng tử nhỏ ($\Delta = \pm 2$), dù sai khác pixel có thể lan qua dự đoán intra và deblocking. Vì cấu trúc cú pháp không đổi, phía nhận liệt kê lại đúng tập ứng viên trên video stego.

```
// native/src/cavlc_stream.cpp (tên hàm ựthc trong ựd án)
decode_cavlc_luma_block(...)
decode_baseline_i_idr_slice(...)
collect_cavlc_trailing_one_sign_candidates(slices)
score_keyed_cavlc_sign_candidate(candidate, key)
select_keyed_cavlc_sign_candidates(candidates, key, required_bits)
patch_annex_b_nal_rbsp(annex_b, nal_index, patches)
```

Bộ thu ứng viên lấy *một ứng viên mỗi khối*: dấu của trailing-one đầu tiên, thuộc mọi loại khối (LumaDC, Luma4x4, ChromaDC, ChromaAC) trong slice IDR. Mỗi ứng viên có định danh ASCII `nal_index:macroblock:category:block_index:rbsp_bit_offset`, ví dụ `24:326:1:6:67801`. Patcher chuyển EBP sang RBP, áp patch độ dài cố định, dựng lại EBP và Annex-B. Gate strict decode xác nhận decoder chấp nhận stream; nó không thay phép đo PSNR/SSIM.

6 Lịch nhúng có khóa, khung kênh và trích xuất mù

6.1 Khóa con HKDF (giao thức v3)

Khóa giấu tin K 32 byte không được dùng trực tiếp làm khóa HMAC. HKDF-SHA256 (RFC 5869) tách nó thành hai khóa con độc lập:

$$\begin{aligned} \text{PRK} &= \text{HMAC-SHA256}(\text{"zkstego-cavlc-v2-salt"}, K), \\ k_{\text{sch}} &= \text{HKDF-Expand}(\text{PRK}, \text{"zkstego/cavlc/v2/schedule"}, 32), \\ k_{\text{wh}} &= \text{HKDF-Expand}(\text{PRK}, \text{"zkstego/cavlc/v2/whitening"}, 32). \end{aligned}$$

Nhân HKDF giữ nguyên từ v2 nên lịch nhúng và dòng làm trắng không đổi; khóa con thứ ba dùng cho thẻ khung của v2 không còn tồn tại. Lộ một khóa con (ví dụ khóa lịch) không làm lộ K hay khóa con còn lại. HMAC ở đây chỉ đóng vai hàm giả ngẫu nhiên có khóa (chọn vị trí, sinh keystream), không dùng để xác thực. Encoder native chỉ giữ khóa lịch và các bit khung đã làm trắng; K của người gọi bị xóa sau HKDF. Khóa K hoàn toàn tách biệt với secret camera s dùng để tạo proof (mục 7).

6.2 Khung kênh v3 và làm trắng

Payload ứng dụng P được bọc thành khung

$$F = [0x03] \parallel [|P|]_{2B, BE} \parallel P, \quad |F| = 3 + |P|, \quad \text{số bit khung} = 8(3 + |P|).$$

Khung không có MAC; overhead chỉ còn 3 byte. Bit nhúng thứ i (MSB trước, đánh số liên tục trên toàn khung kể cả khi trải nhiều IDR) là

$$b_i = F_i \oplus Z_i, \quad Z = \text{HMAC}(k_{\text{wh}}, \text{u64}_{\text{be}}(0)) \parallel \text{HMAC}(k_{\text{wh}}, \text{u64}_{\text{be}}(1)) \parallel \dots,$$

với Z là keystream làm trắng. Không làm trắng thì byte version cố định, trường độ dài nhỏ và cấu trúc proof làm thống kê dấu tại vị trí được chọn bị lệch; sau XOR, các bit nhúng là giả ngẫu nhiên đối với người không có khóa. Đây là lập luận thiết kế; kiểm tra thống kê bậc một trên 27 cấu hình cho $|z| \leq 0.22$ (mục 2), nhưng repository chưa có thực nghiệm steganalysis. Keystream không có nonce: cùng K dùng cho nhiều video sẽ tái sử dụng cùng keystream. Payload tối đa 4096 byte ở các lệnh stđin (giới hạn cấu hình; định dạng cho phép 65,535).

Vì sao bỏ MAC. Giao thức v2 có thêm 16 byte HMAC trên header và payload. Khi Groth16 verify đã bắt buộc và proof ràng buộc message cùng video, thẻ này trùng việc: sửa message hay proof đều làm proof sai, proof tạo bởi camera ngoài sổ đăng ký bị từ chối. Bỏ thẻ tiết kiệm 128 bit (2 IDR ở 64 bit/IDR). Hệ quả: kênh không tự phát hiện bit payload bị sửa, nên *chỉ* bước verify Groth16 xác thực payload. Giao thức v3 không đọc được stego tạo bởi v2 hoặc v1 (byte version khác).

6.3 Lịch theo đoạn IDR

Chỉ có một giao thức, theo *đoạn*, dùng chung cho file, job HTTP và luồng live. Mỗi đoạn là SPS+PPS+một IDR. Với mỗi ứng viên c của đoạn, score là $\text{HMAC-SHA256}(k_{\text{sch}}, \text{id}(c))$; ứng viên được sắp tăng dần theo (score, định danh) và lấy $N = \min(\text{max_bits_per_IDR}, \text{số ứng viên của đoạn})$ vị trí đầu. Bit khung tiếp theo được gán lần lượt cho các vị trí này (bit 0 $\rightarrow$ dấu +, bit 1 $\rightarrow$ dấu -), nên khung trải qua nhiều IDR liên tiếp. Định danh ứng viên tính theo đầu vào của đoạn chứ không theo vị trí trong file, để luồng live và file cho cùng lịch. Giới hạn bit mỗi IDR giới hạn số dấu bị lật trong một frame; bên nhận phải dùng cùng max_bits_per_IDR , nếu không lịch lệch và trích sai. Dịch vụ, CLI và suite benchmark đều dùng mặc định 64.

Ví dụ dung lượng (cấu hình benchmark): message 68 byte tạo payload $4 + 68 + 129 = 201$ byte, khung $3 + 201 = 204$ byte, tức $8 \cdot 204 = 1,632$ bit; ở 64 bit/IDR khung cần $\lceil 1632/64 \rceil = 26$ IDR. Vì bit đã làm trắng giả ngẫu nhiên, chỉ khoảng một nửa số vị trí được lịch chọn thực sự đổi dấu.

6.4 Trích xuất mù

“Mù” nghĩa là bên nhận không cần video gốc hay file phụ (sidecar): (1) parse lại các IDR của stego và liệt kê ứng viên, giống hệt phía gửi vì lật dấu không đổi cấu trúc; (2) dùng cùng K sinh lại khóa con, tính score, đọc bit tại các vị trí đã chọn; (3) bỏ làm trắng 24 bit đầu, kiểm $\text{version} = 3$ và độ dài $\leq$ giới hạn để biết tổng số bit cần đọc — sai khóa làm byte version gần như ngẫu nhiên nên bị từ chối ngay (lọt qua cả version lẫn độ dài với xác suất cỡ $\frac{1}{2^{56}} \cdot \frac{\text{giới hạn}}{65536}$; phần còn sót bị loại khi giải nén proof hoặc khi verify Groth16); (4) đọc đủ khung và bỏ làm trắng để có payload. Payload này *chưa được xác thực*: CLI và job extract trả nó kèm “verified”: false, bên gọi phải unpack và verify Groth16 (job verify làm việc này). Mọi lỗi (version sai, độ dài vượt giới hạn, thiếu capacity, hết video trước khi đủ khung, lỗi parse/I/O) đều thoát mã 2. Tính đúng của lịch, khung, làm trắng và lịch theo đoạn giữa C++ và Python được kiểm bằng phase 2 của bộ run_all.py.

7 Groth16: sổ đăng ký camera, ràng buộc video và mạch

Proof chứng minh phát biểu: “*một camera nằm trong sổ đăng ký (root Merkle công khai) xác nhận đúng video này và message này*”, trong khi bên kiểm chứng không cần secret của camera và không biết camera nào. Hai khóa được tách hẳn: khóa giấu tin K (32 byte, người gửi và bên kiểm chứng cùng giữ; chọn vị trí và làm trắng) và secret camera s (phần tử trường BN254, chỉ camera giữ; dùng để tạo proof).

7.1 Sổ đăng ký camera

src/camera_registry.py dựng cây Merkle nhị phân độ sâu 16 (tối đa $2^{16} = 65,536$ camera), nút Poseidon(trái,phải), lá trống bằng 0, lá i là $pk_i = \text{Poseidon}(s_i)$. Poseidon được cài bằng Python (src/poseidon.py, hằng số của circomlib) và đối chiếu với vector chuẩn của circomlibjs cũng như với witness của chính mạch. Bên kiểm chứng chỉ tin *root* của cây; thêm hoặc thu hồi camera nghĩa là công bố root mới.

7.2 Hash video và binding

Lệnh zkstego_blind_bits video-digest (bản tham chiếu src/video_binding.py) nhận video, số bit khung n_F và cap, đọc khóa K từ stdin vì cần định vị các bit carrier. Với $n_F = 8(3 + |P|)$ biết trước khi tạo proof, *carrier* là đúng n_F vị trí dấu mà lịch theo khóa sẽ ghi khung vào (mỗi IDR $\min(\text{cap}, |C|)$ vị trí đầu của lịch, cho tới hết khung). Khi đó

$$\begin{aligned} D &= \text{SHA256} \left(\text{"zkstego/video-digest/v1"} \parallel \text{u32}(n_F) \parallel \text{u32}(\text{cap}) \right. \\ &\quad \left. \parallel \parallel_{\text{NAL}} \text{u32}(1 + |r'|) \parallel \text{hdr} \parallel r' \right), \\ \beta &= \text{SHA256} \left(\text{"zkstego/proof-binding/v1"} \parallel \text{mode} \parallel D \parallel \text{SHA256}(m) \right), \\ (\beta_H, \beta_L) &= (\beta[0:16], \beta[16:32]), \end{aligned}$$

trong đó r' là RBSP của từng NAL (đã bỏ byte EPB) với *chỉ* các bit carrier đặt về 0, và β_H, β_L là hai số nguyên big-endian 128 bit (nhỏ hơn modulus trường BN254). Nhưng chỉ đổi đúng các bit carrier (byte EPB có thể thay đổi nhưng không ảnh hưởng RBSP), nên $D(\text{cover}) = D(\text{stego})$. Mọi thay đổi khác — video khác, cắt hoặc thêm frame, sửa một bit ở frame sau, lật một dấu không mang khung, mã hóa lại — làm D đổi và proof sai. Lật một bit carrier giữ nguyên D nhưng làm payload trích ra đổi, nên proof cũng bị từ chối. Vì vị trí carrier phụ thuộc K , chỉ người có khóa giấu tin mới tính được D ; bên kiểm chứng vốn đã cần K để trích xuất.

Chế độ $\text{mode} = 0$ ràng buộc video; $\text{mode} = 1$ chỉ ràng buộc message (D là 32 byte 0) và dành cho luồng live, vì khi proof được nhúng thì phần còn lại của video chưa tồn tại. Job verify từ chối $\text{mode} = 1$ với lý do video_binding_required, trừ khi bật ZK_STEGO_ALLOW_MESSAGE_ONLY_PROOFS=1.

7.3 Payload

Payload ứng dụng là

$$P = [0x01] \parallel [\text{mode}]_{1B} \parallel [|m|]_{2B, BE} \parallel m \parallel \pi, \quad |P| = 4 + |m| + 129,$$

trong đó π là proof Groth16 nên 129 byte. Với giới hạn payload 4096 byte, message tối đa $4096 - 4 - 129 = 3963$ byte; khung tương ứng là $3 + 4096 = 4099$ byte, tức 32,792 bit.

7.4 Quan hệ được chứng minh

Mạch `circuits/camera_video.circom` (`CameraVideo(16)`) có public input $x = (\text{root}, \beta_H, \beta_L)$ và private witness $w = (s, \sigma_0, \dots, \sigma_{15}, \iota_0, \dots, \iota_{15})$ (secret camera, các nút anh em và chỉ số hướng trên đường Merkle). Quan hệ là

$$\mathcal{R} = \left\{ (x, w) : \iota_j(\iota_j - 1) = 0 \ \forall j, \quad n_0 = \text{Poseidon}(s), \quad n_{16} = \text{root} \right\},$$

với $n_{j+1} = \text{Poseidon}(L_j, R_j)$, $L_j = n_j + \iota_j(\sigma_j - n_j)$, $R_j = \sigma_j + \iota_j(n_j - \sigma_j)$. Hai public input β_H, β_L không tham gia ràng buộc nào khác nên mạch thêm hai ràng buộc β_H^2, β_L^2 để gắn chúng tường minh vào R1CS (cách làm của Semaphore). Mạch có 8,737 constraints và 3 public input. Bên kiểm chứng chấp nhận khi và chỉ khi

$$\text{Groth16.Verify}(\nu k, (\text{root}_{\text{tin cây}}, \beta_H, \beta_L), \pi) = 1,$$

với β được *tính lại* từ message trích ra và từ hash D của chính file video nhận được. Theo tính knowledge soundness của Groth16 và tính không biến đổi được proof sang public input khác (weak simulation-extractability), người không biết s của một lá trong cây không tạo được proof cho binding mới; proof tạo cho một binding (video + message) hay một root khác không verify được. Hash SHA-256 được tính ngoài mạch, nên mạch nhỏ hơn mạch SHA-256 cũ khoảng 7 lần.

Proof chứng minh: một camera có khóa trong sổ đăng ký đã xác nhận đúng video (mọi bit; các bit carrier được bảo vệ gián tiếp qua payload) và đúng message; bên kiểm chứng không biết camera nào và không giữ secret nào đủ để làm giả proof. **Proof không chứng minh:** camera thật sự quay cảnh đó (ví dụ quay lại màn hình), thời điểm quay, hay tính realtime; với luồng live (mode = 1) proof chỉ ràng buộc message.

7.5 Các hàm, artifact và setup

`CameraProofBridge` trong `src/zk_proof.py` tính witness bằng WASM của mạch rồi gọi `snarkjs groth16 prove` với `camera_video.zkey`. Proof gồm $A \in G_1, B \in G_2, C \in G_1$ trên BN254, nên còn 129 byte (tọa độ x của A, B (hai phần tử $\mathbb{F}_{p^2}$), C , mỗi phần 32 byte, cộng 1 byte cò chứa 3 bit chẵn/lẻ của y). `bytes_to_proof` kiểm tọa độ $< p$ và điểm nằm trên đường cong; dữ liệu hỏng bị từ chối với lý do `malformed_proof_payload`. Verify chỉ chấp nhận khi `snarkjs` thoát 0 và in OK!.

```
# òLung ửgi/ậnhn ệhin ạti (README, src/camera_proof.py)
bridge = CameraProofBridge("circuits")
signed = build_payload(bridge, registry, camera_secret, message,
    native_cli=cli, cover="cover.h264", stego_key=K) # hash cover -> binding ->
    prove
# stdin: K hex, signed.payload hex
# zkstego_blind_bits embed-stream-auth-stdin cover.h264 stego.h264 64
# zkstego_blind_bits extract-stream-auth stego.h264 - <max_payload> 64 -> payload (verified:
    false)
verdict = verify_payload(bridge, registry.root, payload,
    native_cli=cli, stego="stego.h264", stego_key=K) # hash stego -> binding
    -> verify
if not verdict.video_bound:
    raise ValueError(verdict.reason) # e.g. proof_invalid
```

Khóa không bao giờ đi qua tham số dòng lệnh: CLI đọc khóa dạng một dòng 64 ký tự hex (và payload hex) từ stdin. Hậu tố `-auth` của tên lệnh được giữ lại nhưng nay mang nghĩa “có khóa”, không phải có MAC.

Setup (py -3.12 -m src.zk_setup): Phase 1 là Powers of Tau công khai của Hermez (2^{16} , tệp `powersOfTau28_hez_final_16.ptau`, hash BLAKE2b được ghim); Phase 2 gồm hai lượt đóng góp (entropy qua stdin), một beacon ngẫu nhiên sinh cục bộ, rồi `snarkjs zkey verify`; transcript ở `circuits/build/camera_video_setup.json`. Vì mọi bước chạy trên một máy nên đây là nghi thức *demo*; triển khai thật cần nhiều bên đóng góp độc lập (chỉ cần một bên trung thực). Setup chạy một lần cho mạch, không lặp lại theo video; proof tạo bằng khóa của mạch cũ không verify với khóa hiện tại.

8 API và kết nối

8.1 HTTP file jobs

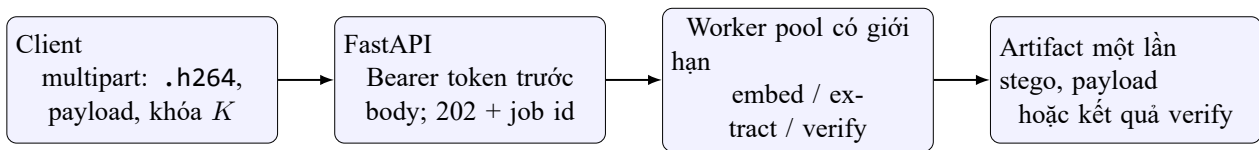


Figure 10: HTTP file flow. Client upload không phải kết nối camera trực tiếp; job chạy trong worker và artifact được tải khi hoàn tất.

Một ứng dụng duy nhất (`src.api.native_handlers:create_native_app`) dựng trên lõi job dùng chung. Embed gửi multipart `.h264` cùng chuỗi byte cần nhúng (`message_b64`, thường là payload camera đã ký sẵn ở phía camera) và khóa `K` tới `/api/v1/jobs/embed`; extract gửi stego và khóa tới `/api/v1/jobs/extract` và nhận payload kèm `"verified": false`; service trả job id, client poll `/api/v1/jobs/{id}` rồi tải artifact một lần (hết hạn sau 10 phút).

POST `/api/v1/jobs/verify` là job duy nhất xác thực: trích xuất mù → unpack → hash video của chính file tải lên → Groth16 verify *bắt buộc* với root của sổ đăng ký (`ZK_STEGO_CAMERA_REGISTRY`; thiếu cấu hình này job thất bại). Trạng thái `succeeded` kèm message và cờ `video_bound` chỉ khi proof đúng; ngược lại `rejected` với lý do `payload_not_found`, `malformed_proof_payload`, `video_binding_required`, `video_digest_unavailable` hoặc `proof_invalid`; lỗi hạ tầng (thiếu sổ đăng ký, khóa, Node.js) là `failed`. Trạng thái job gồm `queued`, `running`, `succeeded`, `rejected`, `failed`; client không được coi `rejected` là thành công.

Bảo vệ: Bearer token (tối thiểu 32 ký tự) được kiểm tra trước khi đọc body; 10 lần xác thực sai từ một IP trong 60 s trả 429 kèm `Retry-After`; tối đa 4 upload đồng thời; giới hạn kích thước và thời gian upload; hàng đợi job có giới hạn; job xong bị xóa sau 24 giờ; tiến trình native được gọi không qua shell, có timeout, khóa đi qua stdin.

8.2 WebSocket camera stream

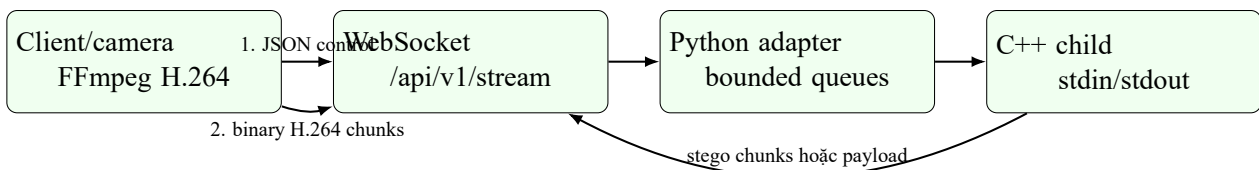


Figure 11: WebSocket stream là giao thức điều khiển + media chunks; output embed là binary stream, còn extract trả payload (chưa xác thực) qua JSON.

Client mở WebSocket có Bearer authorization (handshake thiếu token bị từ chối trước khi accept) rồi gửi JSON control: thao tác, khóa và, khi embed, payload; extract gửi giới hạn độ dài payload. Sau phản hồi ready, client gửi từng H.264 chunk dạng binary; adapter chuyển sang stdin của tiến trình C++ (embed-live-auth-stdin / extract-live-auth-stdin) theo backpressure với ngưỡng 64 KiB high-water/16 KiB low-water. Khi embed, output native được gửi lại dạng binary chunks; khi extract, payload trả JSON Base64 kèm "verified": false. Luồng đóng sau 15 s không có dữ liệu.

```
# src/api/native_handlers.py - quy trình stream (ước l)
start = json.loads(control_text)
process = await asyncio.create_subprocess_exec(
    executable, *arguments, stdin=PIPE, stdout=PIPE, stderr=PIPE)
await websocket.send_json({"type": "ready", "operation": operation})
# binary websocket chunks -> native stdin; native stdout -> websocket
await process.stdin.drain() # backpressure
await websocket.send_bytes(chunk) # embed output
```

Luồng WebSocket không tự chạy Groth16; client phải unpack và verify proof (như listing ở mục 7) hoặc dùng job verify. Vì proof của luồng live được tạo trước khi phần còn lại của video tồn tại, nó chỉ có thể ở mode = 1 (ràng buộc message), và job verify mặc định từ chối chế độ này. Luồng không tạo TLS hay camera attestation.

9 Phân tích bảo mật, mô hình đe dọa và giới hạn

Mô hình. Bên kiểm chứng tin root của sổ đăng ký và verification key, và giữ khóa giấu tin K ; secret camera chỉ nằm trên camera. Kẻ tấn công có thể sửa, cắt, ghép, mã hóa lại stream, chép payload sang video khác, hoặc chính là người có K (ví dụ một bên nhận) muốn thay message.

Table 10: Tính chất bảo mật của hệ thống hiện tại và cơ chế bảo đảm.

Tính chất	Cơ chế và bằng chứng
Xác thực nguồn gốc	Chỉ camera biết s của một lá trong cây có root tin cây mới tạo được proof; camera ngoài sổ đăng ký bị từ chối (unregistered_camera).
Ấn danh trong sổ đăng ký	Đường Merkle và s là witness riêng; bên kiểm chứng không biết lá nào đã được dùng.
Không cần secret ở bên kiểm chứng	Bên kiểm chứng chỉ cần root, verification key và K ; giữ K không đủ để làm giả proof (message_swap bị từ chối).
Ràng buộc video	D phủ mọi NAL trừ bit carrier; phát lại sang video khác, cắt ngắn, sửa frame sau, lật dấu ngoài lịch đều làm proof sai (ma trận 11/11).
Toàn vẹn payload	Không có MAC; sửa bit carrier làm payload đổi nên proof sai hoặc hỏng (flip_scheduled_sign).
Bí mật vị trí	Lịch HMAC theo khóa con và làm trắng bit; sai khóa không tìm thấy khung (wrong_key).

Table 11: Giới hạn bảo mật và phạm vi của hệ thống hiện tại.

Giới hạn	Hệ quả
Luồng live chỉ ràng buộc message	Ở mode = 1 proof không phủ video; job verify từ chối trừ khi bật ZK_STEGO_ALLOW_MESSAGE_ONLY_PROOFS=1.
Trusted setup chỉ cho demo	Phase 1 là Powers of Tau công khai, nhưng Phase 2 (hai lượt đóng góp và beacon) chạy trên một máy; triển khai cần nhiều bên độc lập.
Không chứng minh cảnh thật	Proof không chứng minh camera thật sự quay cảnh đó (ví dụ quay lại màn hình) hay thời điểm quay.
Kiểm chứng cần khóa giấu tin	D phụ thuộc vị trí carrier, tức phụ thuộc K ; chỉ người có K mới kiểm chứng được (bên kiểm chứng vốn cần K để trích xuất).
Keystream không nonce	Cùng K cho nhiều video tái sử dụng cùng keystream làm trắng.
Chưa chứng minh tính khó phát hiện	Chỉ có kiểm tra thống kê bậc một ($ z \leq 0.22$); chưa đo steganalysis.
Profile H.264 hẹp	Chỉ Baseline/CAVLC, progressive, 4:2:0, một slice/frame bắt đầu ở MB 0, không I_PCM, một SPS/PPS id; chỉ IDR mang dữ liệu; không CABAC, không B-frame.
Mỏng manh có chủ đích	Mọi lần mã hóa lại phá payload (transcode bị từ chối); hệ thống không chống biến đổi.
Tương thích phiên bản	v3 không đọc stego v2/v1; proof theo khóa mạch cũ không verify với khóa hiện tại.
Realtime và edge	Chưa nghiệm thu: camera E2E 7.608 FPS so với cổng 30 FPS (giao thức v1, đo 24/09/2026, trước bản sửa native 02/10/2026); chưa có số đo trên thiết bị edge.

10 Kết luận và giới hạn

Hệ thống hiện tại hợp nhất về một lõi media native C++ với giao thức kênh v3 (hai khóa con HKDF, khung `[0x03][len 2 B][payload]` không MAC, làm trắng bit, lịch theo đoạn IDR với 64 bit/IDR), trích xuất mù chỉ với khóa và Groth16 verify bắt buộc trên mạch camera_video (8,737 constraints, 3 public input). Proof ràng buộc camera trong sổ đăng ký với đúng video và đúng message; bên kiểm chứng chỉ cần root, verification key và khóa giấu tin, không cần secret camera và không biết camera nào. Về chức năng, phiên bản này có bằng chứng từ bộ test ngày 04/10/2026 (142/142 qua 12 phase, CTest 1/1), không gồm camera vật lý.

Số đo ngày 04/10/2026 cho media 27/27 (mọi proof ràng buộc video, 0 dấu đổi ngoài lịch, frame tệ nhất 44.03 dB, 0/27 dưới 40 dB), đầu-cuối 5/5 (gửi 2.78 s, nhận 1.99 s trên clip CIF 300 frame) và ma trận tấn công 11/11 đúng kỳ vọng, gồm từ chối phát lại sang video khác, cắt ngắn, thay message và camera ngoài sổ đăng ký. Chi phí bị chi phối bởi parse bitstream (khoảng 61 / 245 / 891 ms mỗi IDR mang khung ở ba độ phân giải), không phải Groth16; camera vật lý chỉ có số liệu lịch sử 7.608 FPS (giao thức v1), không đạt cổng 30 FPS. Chưa có cơ sở khẳng định hỗ trợ P/B-frame mang dữ liệu, CABAC, realtime, hay khả năng chống steganalysis.

Ưu tiên tiếp theo: đo lại camera vật lý với giao thức v3 và proof camera, giảm chi phí parse native, ràng buộc video cho luồng live (ví dụ proof theo đoạn), nghi thức Phase 2 nhiều bên độc lập, nonce cho keystream và thực nghiệm steganalysis. So sánh với hệ khác cần cùng video, QP, payload, độ phân giải và điều kiện host.

Artifact được dùng và kết quả đã loại bỏ

- Media run 20261004T142448Z_1db00a (giao thức v3, proof camera): video_pipeline_new.json, quality_per_frame_new.csv.
- Đầu-cuối và ma trận tấn công: e2e_new.json; proof system: zkp_new.json; báo cáo tổng hợp benchmark_report_new.pdf.
- Camera (số liệu lịch sử, giao thức v1): realtime_camera_runs_20260924.json, run 24.

- Quy trình chạy: `benchmark/NEW_BENCHMARKS.md`; mô tả hệ thống: `doc/he_thong_hoat_dong.md`; trạng thái nghiệm thu: `doc/completion_plan.md`.
- *Đã loại bỏ khỏi báo cáo (lịch sử, không phải bằng chứng hiện tại)*: pipeline Python cũ và các kết quả SEC1–SEC10 của nó; mạch SHA-256 cũ cùng khung kênh v2 có thẻ MAC; số đo media và ZKP ngày 24/09/2026 của giao thức v1. Mã này chỉ còn trong lịch sử git.